Programa:	Search and Recover 3
Protección:	ASPack + Time-Trial
Objetivos:	Desempakar y encontrar un serial válido
URL descarga:	www.iolo.com
Dificultad:	Muy fácil
Herramientas:	OllyDbg 1.10 + PEiD y/o RDG packer + Import Reconstructor.
Cracker:	SyXe'05 ... [hAcK-c0d3d] ... [CracksLatinos] Fecha: 02-10-2005

hola a tod@s una vez más..!!

Hoy tenía un ratito libre y pensé en aprovecharlo escribiendo algo, esta vez topé con algo realmente sencillo, no hace mucho hice una versión anterior que era igual de fácil, aún así me he decidido a compartirlo con vosotros, estudiaremos el código y la rutina de protección del programa que deberemos descubrir, y lo haremos de una manera muy sutil ;-)

Antes de nada instalamos el programa, arriba en las indicaciones añado el link de la página donde descargarlo, de todos modos si cambian la versión o cualquier cosa me lo podéis pedir que yo lo tengo aquí y os lo paso, una vez instalado lo miramos con PEiD o RDG packer que va muy bien:

ASPack 2.12b -> Alexey Solodovnikov

Sencillito, es un ASPack 2.12b este packer no presenta ninguna complicación, incluso hay muchos scripts escritos para localizar el OEP fácilmente, de todos modos por si alguien no lo sabe diré como hacerlo mediante el valor de ESP, abrimos el programa en Olly y vemos:

```
01597001   90                NOP
01597002   60                PUSHAD        <-- Esta instrucción nos interesa..!!
01597003   E8 03000000       CALL SearchAn.0159700B
01597008 - E9 EB045D45       JMP 46B674F8
0159700D   55                PUSH EBP
0159700E   C3                RETN
```

Pulsamos F7/F8 igual da hasta superar el PUSHAD marcado, en este momento miramos en la ventana de registros y vemos esto:

```
Registers (3DNow!)
EAX 00000000
ECX 0012FFB0
EDX 7FFE0304
EBX 7FFDF000
ESP 0012FFA4   <-- Este interesa..!!
EBP 0012FFF0
ESI 77D15CBC user32.77D15CBC
EDI 77D15CCA user32.77D15CCA
EIP 01597003 SearchAn.01597003
```

Esto es lo que queda, vemos que ESP quedó en rojo, este es el que seguiremos, picamos sobre él con el botón derecho y elegimos 'Follow in Dump' para que nos muestre su contenido en la ventana Dump (ventana de datos), una vez allí le ponemos un BreakPoint tipo Hardware en Acceso (On Access) si lo ponemos BPM normal no corre por esto lo hacemos así de este modo.

Una vez puesto el HBP en ESP damos RUN y rápidamente para aquí:

```
015973AA   8985 A8030000   MOV DWORD PTR SS:[EBP+3A8],EAX
015973B0   61              POPAD
015973B1 v 75 08           JNZ SHORT SearchAn.015973BB
015973B3   B8 01000000     MOV EAX,1
015973B8   C2 0C00         RETN 0C
015973BB   68 0C0A7600     PUSH SearchAn.00760A0C
015973C0   C3              RETN
```

Se detiene justo antes del salto al OEP, anotamos el valor:

OEP:760A0C

Realmente el ASPack es muy simple y no debe dar problemas, busquemos la IAT para que el Import nos repare el dumpeado que vamos a hacer, entramos en el primer CALL que vemos debajo del OEP y entramos también en el primer SUBCALL que encontramos para llegar a:

```
00407BBC - FF25 4803F700   JMP DWORD PTR DS:[F70348]   kernel32.GetModuleHandleA
00407BC2   8BC0            MOV EAX,EAX
00407BC4 - FF25 4403F700   JMP DWORD PTR DS:[F70344]   kernel32.LocalAlloc
00407BCA   8BC0            MOV EAX,EAX
00407BCC - FF25 4003F700   JMP DWORD PTR DS:[F70340]   kernel32.TlsGetValue
00407BD2   8BC0            MOV EAX,EAX
```

Estupendo, los saltos a la IAT, elegimos el primero de ellos y hacemos un 'Follow in dump->memory Address' para caer en medio de la tabla, subimos un poco y encontramos el inicio:

```
Address   Hex dump                                          ASCII
00F70244  CA 25 F4 77 90 56 F6 77 DE 55 F6 77 45 A7 E5 77
00F70254  CB 15 E6 77 72 AC E5 77 A0 60 E5 77 82 A6 E5 77
00F70264  9B A2 E5 77 7F 61 E5 77 42 D1 E5 77 DF A7 E5 77
00F70274  71 A6 E5 77 60 A6 E5 77 44 7F E5 77 49 A9 E5 77
00F70284  AF 9B E4 77 FC A7 E5 77 E1 60 E5 77 4E 27 E5 77
```

Lo anotamos también claro:

inicio iat: F70244

Y hacemos lo mismo con el final que encontramos en:

Address	Hex dump	ASCII
00F70CB4	D8 1A B0 76 2C 8F B0 76 6B 8E B0 76 33 84 B1 76	ï→▒v,A▒vkA▒v3▒▒v
00F70CC4	00 00 00 00 6B 7D D3 77 BB 91 D5 77 00 00 00 00	....k}Ew╗æ'w....
00F70CD4	71 E4 3B 00 00 00 00 00 A1 39 E5 77 00 00 00 00	qä;.....í9åw....
00F70CE4	D7 4D 3E 77 68 16 3E 77 00 00 00 00 60 3F C5 77	ïM>wh►>w....`?A
00F70CF4	00 00 00 00 6B 65 72 6E 65 6C 33 32 2E 64 6C 6C	....kernel32.dll

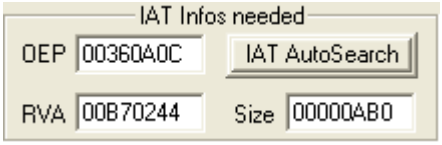
Ya lo tenemos también, nos lo anotamos:

final iat: F70CF4

De estos dos valores obtenemos su longitud:

longitud iat: AB0

Una vez que tenemos ubicado el OEP y la IAT bien delimitada dumpeamos el archivo con el mismo plugin de Olly, no me cansaré de repetir que a mi me va muy bien ;-DD Las opciones serán las de siempre, destildamos el 'Rebuild Import' y tildamos en 'Fix Raw Size & Offset...' picamos en 'Get EIP as OEP' y luego en 'Dump' le damos un nombre y listos, yo lo llamé dump.exe como siempre hago para no complicarme mucho, después abrimos el Import y elegimos el proceso que encontraremos en la lista desplegable en 'Attach to an Active Process' y le damos los valores correctos:



Como siempre decir que trabajamos con offsets en este programa, de ahí que el OEP esté en :760A0C y pongamos :360A0C, este valor se obtiene como sabemos de restarle el imagebase=400000, sería: 760A0C - 400000 = 360A0C (Lo mismo ocurre con el campo RVA que marca el inicio de la IAT).

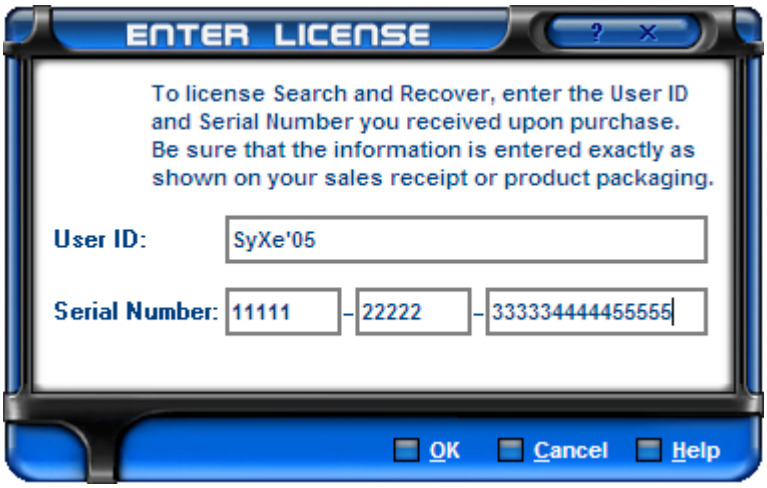
hecho esto seleccionamos 'Get Imports' para que nos encuentre las funciones importadas del programa, cuando ya las tenemos picamos en 'Fix Dump' y en el pequeño explorador que se abre seleccionamos el dumpeado dump.exe en mi caso, nos dejará un ejecutable dump_.exe justo al lado que ya podemos ejecutar sin ASPack.

Como hemos visto quitar este packer es de lo más sencillo que nos podemos encontrar, he preferido darle un ligero repaso al método por si alguien todavía no lo ha puesto en práctica, bien porque acaba de empezar con esto o por cualquier otro motivo.

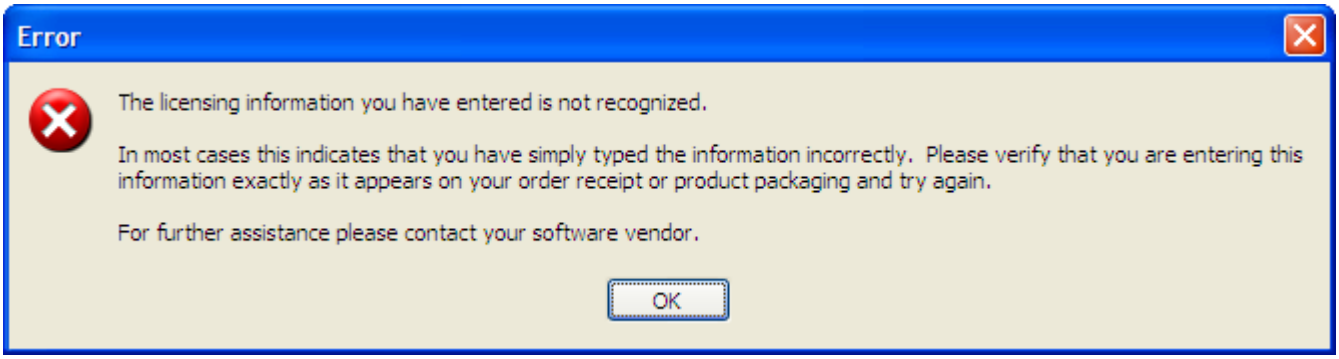
Y bien una vez desempaquetado podemos meternos con él tranquilamente, ahora los bytes que encontremos en Olly estarán físicamente en el ejecutable, al contrario de lo que sucedía al estar comprimido que no existían, esto es bueno por si tenemos que parchear en alguna dirección, iríamos al offset correspondiente a ella y modificaríamos lo necesario. Cargamos de momento el dump_.exe en Olly y damos RUN a ver que sucede:

Pues ya ves que nos sale una pedazo de nag, jeje.. vale nos da la opción de Introducir un serial, hmmmm... esto me gusta, de continuar el Trial, de Salir y de pedir Ayuda ¿?

Trataremos antes de introducir un código de registro a ver que ocurre, jejeje...seleccionamos donde dice 'Enter Serial Number' y nos aparece el cuadro de registro, rellenamos los campos con nuestro nombre y un serial que nos inventamos por su puesto:

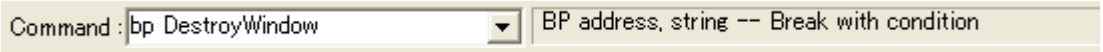


Cuando los tenemos puestos presionamos en el botón 'OK' de esta manera los datos serán recogidos y verificados, al momento obtenemos la ingrata respuesta (esperada claro):



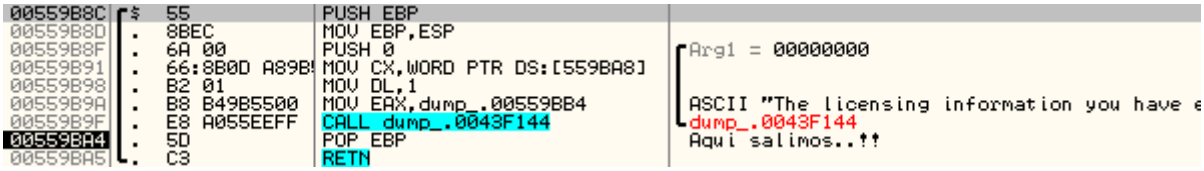
un pedazo de mensaje eh! joder no escatiman nada, por supuesto que nos va a aservir de ayuda para localizar el momento en que nuestros datos son validados, y una vez allí descubrir el serial válido ;-) si es que de todo lo malo sale algo bueno, jeje..

Pues nada al ataque se ha dicho, supuestamente podría ser el típico MesasgeBox pero si ponemos un BP en esta API no para ¿? lo mismo para otros BP's que puse y no sirvieron, en estos casos suelo recurrir a una API que se llama para destruir la ventana, si DestroyWindow ya lo has adivinado, entonces no se hable más, con el mensaje en pantalla ponemos el BP:



Damos al Enter y tenemos el BP puesto, aceptamos el mensaje y Olly para aqui:

Pero esto no es lo que buscamos está claro, deberemos ir hacia atrás bien pulsando Ctrl+F9 bien traceando con F8, la cuestión es llegar a esta parte:



Esto sí está mejor, jeje.. vemos la cadena del mensaje de error, el 'PUSH EBP' marca el inicio de la zona y es referenciada por un CALL como marca el símbolito $ que tiene delante. Hasta aquí correcto pero hay algo todavía mejor, si llegamos hasta el RET y lo superamos caemos justo en la zona del mensaje de chico_bueno como se suele decir:

```
0075B29F  .  6A 00          PUSH 0                                  Arg1 = 00000000
0075B2A1  .  66:8B0D 04B3   MOV CX,WORD PTR DS:[75B304]
0075B2A8  .  B2 02          MOV DL,2
0075B2AA  .  B8 10B37500    MOV EAX,dump_.0075B310                  ASCII "Thank you for licensing Search and F
0075B2AF  .  E8 903ECEFF    CALL dump_.0043F144                     dump_.0043F144
0075B2B4  .  C786 4C02000(  MOV DWORD PTR DS:[ESI+24C],1
0075B2BE  .v EB 05          JMP SHORT dump_.0075B2C5
0075B2C0  >  E8 C7E8DFFF    CALL dump_.00559B8C
0075B2C5  >  33C0           XOR EAX,EAX                             Aqui salimos..!!
0075B2C7  .  5A             POP EDX
```

Entonces debe haber un salto que nos envíe hacia el mensaje de ERROR que vimos antes, subiendo un pelín con la ruedecilla del ratón encontramos dicho salto:

```
0075B264  .  E8 9B5ED2FF    CALL dump_.00481104
0075B269  .  8B55 E4        MOV EDX,DWORD PTR SS:[EBP-1C]
0075B26C  .  A1 00AE7600    MOV EAX,DWORD PTR DS:[76AE00]
0075B271  .  8B00           MOV EAX,DWORD PTR DS:[EAX]
0075B273  .  8B4D FC        MOV ECX,DWORD PTR SS:[EBP-4]
0075B276  .  E8 F9CBD4FF    CALL dump_.004A7E74                     Bonita llamada..!!
0075B27B  .  84C0           TEST AL,AL
0075B27D  .v 74 41          JE SHORT dump_.0075B2C0                 Este es el SALTO..!!
0075B27F  .  8D55 E0        LEA EDX,DWORD PTR SS:[EBP-20]
0075B282  .  8B86 1C03000(  MOV EAX,DWORD PTR DS:[ESI+31C
0075B288  .  E8 775ED2FF    CALL dump_.00481104
0075B28D  .  8B55 E0        MOV EDX,DWORD PTR SS:[EBP-20]
0075B290  .  A1 00AE7600    MOV EAX,DWORD PTR DS:[76AE00]
0075B295  .  8B00           MOV EAX,DWORD PTR DS:[EAX]
0075B297  .  8B4D FC        MOV ECX,DWORD PTR SS:[EBP-4]
0075B29A  .  E8 4DCCD4FF    CALL dump_.004A7EEC
0075B29F  .  6A 00          PUSH 0                                  Arg1 = 00000000
0075B2A1  .  66:8B0D 04B3   MOV CX,WORD PTR DS:[75B304]
0075B2A8  .  B2 02          MOV DL,2
0075B2AA  .  B8 10B37500    MOV EAX,dump_.0075B310                  ASCII "Thank you for licensing Search and F
0075B2AF  .  E8 903ECEFF    CALL dump_.0043F144                     dump_.0043F144
0075B2B4  .  C786 4C02000(  MOV DWORD PTR DS:[ESI+24C],1
0075B2BE  .v EB 05          JMP SHORT dump_.0075B2C5
0075B2C0  >  E8 C7E8DFFF    CALL dump_.00559B8C
0075B2C5  >  33C0           XOR EAX,EAX                             Aqui salimos..!!
0075B2C7  .  5A             POP EDX
0075B2C8  .  59             POP ECX
0075B2C9  .  59             POP ECX
0075B2CA  .  64:8910        MOV DWORD PTR FS:[EAX],EDX
0075B2CD  .  68 EFB27500    PUSH dump_.0075B2EF
0075B2D2  >  8D45 E0        LEA EAX,DWORD PTR SS:[EBP-20]
0075B2D5  .  BA 07000000    MOV EDX,7
0075B2DA  .  E8 1DA1CAFF    CALL dump_.004053FC
0075B2DF  .  8D45 FC        LEA EAX,DWORD PTR SS:[EBP-4]
0075B2E2  .  E8 F1A0CAFF    CALL dump_.004053D8
0075B2E7  .  C3             RETN
```

he ampliado la imagen para que muestre el salto que he remarcado en rojo, pero antes tenemos un CALL donde presumiblemente se decide si el serial es válido o no, nada ponemos un BP sobre él:g

```
0075B276  .  E8 F9CBD4FF    CALL dump_.004A7E74                     Bonita llamada..!!
0075B27B  .  84C0           TEST AL,AL
0075B27D  .v 74 41          JE SHORT dump_.0075B2C0                 Este es el SALTO..!!
0075B27F  .  8D55 E0        LEA EDX,DWORD PTR SS:[EBP-20]
```

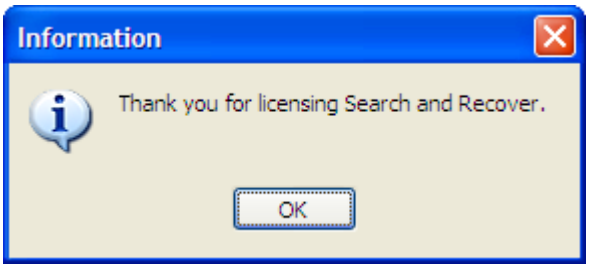
Y volvemos a introducir los datos de registro, bueno no hace falta porque quedaron puestos sóo hay que pulsar 'OK' y paramos en el BP, si miramos en los registros vemos en ECX un puntero al serial introducido, interesante ;-) y en EDX un puntero al nombre, o sea que lo que deba suceder será dentro del CALL, entremos con F7 a ver:

```
004A7EA6  .v 74 1B          JE SHORT dump_.004A7EC3
004A7EA8  .  8D4D F8        LEA ECX,DWORD PTR SS:[EBP-8]
004A7EAB  .  8A53 04        MOV DL,BYTE PTR DS:[EBX+4]
004A7EAE  .  8BC6           MOV EAX,ESI
004A7EB0  .  E8 A3F0FFFF    CALL dump_.004A6F58
004A7EB5  .  8B55 F8        MOV EDX,DWORD PTR SS:[EBP-8]
004A7EB8  .  8BC7           MOV EAX,EDI
004A7EBA  .  E8 41D9F5FF    CALL dump_.00405800                     Este decide todo..!!
004A7EBF  .  0F9445 FF      SETE BYTE PTR SS:[EBP-1]
004A7EC3  >  33C0           XOR EAX,EAX
004A7EC5  .  5A             POP EDX
004A7EC6  .  59             POP ECX
004A7EC7  .  59             POP ECX
004A7EC8  .  64:8910        MOV DWORD PTR FS:[EAX],EDX
004A7ECB  .  68 E07E4A00    PUSH dump_.004A7EE0
004A7ED0  >  8D45 F8        LEA EAX,DWORD PTR SS:[EBP-8]
004A7ED3  .  E8 00D5F5FF    CALL dump_.004053D8
004A7ED8  .  C3             RETN
004A7ED9  .^ E9 72CDF5FF    JMP dump_.00404C50
004A7EDE  .^ EB F0          JMP SHORT dump_.004A7ED0
004A7EE0  .  8A45 FF        MOV AL,BYTE PTR SS:[EBP-1]
004A7EE3  .  5F             POP EDI
004A7EE4  .  5E             POP ESI
004A7EE5  .  5B             POP EBX
004A7EE6  .  59             POP ECX
004A7EE7  .  59             POP ECX
004A7EE8  .  5D             POP EBP
004A7EE9  .  C3             RETN
```

Allí remarco lo que ví más decisivo de todo, al final de la rutina vemos que AL toma su valor de la variable local [EBP-1] y esta toma su valor en :4A7EBF en una instrucción tipo SETE que lo que hace es poner la variable a 1 en caso que el FLAG_ZERO esté activado y a 0 en caso contrario, como sabemos que el retorno de la función deberá ser AL==1 deducimos que [EBP-1] deberá ponerse a 1 ya que le pasa su valor, y entonces el CALL deberá activar el FLAG_ZERO, es una sucesión de hechos, o sea, que todo se decidirá dentro de este CALL que hay en :4A7EBA, si traceamos hasta llegar a él y miramos los registros tenemos el premio ya por adelantado ;-)

```
Registers (FPU)
EAX 01D46184 ASCII "11111-22222-333334444455555"
ECX 0012F61C
EDX 01D45690 ASCII "61656-R3980-5727967808"  <---- Serial OK..!!
EBX 01BA11B8
ESP 0012F698
EBP 0012F6B8
ESI 01D4722C ASCII "SyXe'05"
EDI 01D46184 ASCII "11111-22222-333334444455555"  <---- Nuestro Serial..!!
EIP 004A7EBA dump_.004A7EBA
```

Pues ahora sí parece que encontramos lo que buscábamos, tiene la pinta de ser el serial válido pero mucho, nada a probarlo que es gratis, jeje..



Pues nada, un sencillo ejercicio de ingeniería inversa.

Y como no da para más el tute terminamos, la verdad que encontré el programa en la red y me pareció hacer el tute, pues se resuelve muy muy fácil lo que alienta sobretodo a los que están empezando y necesitan practicar.

Para cualquier cuestión ya saben:

syxe00@yahoo.es
syxe0@hotmail.com